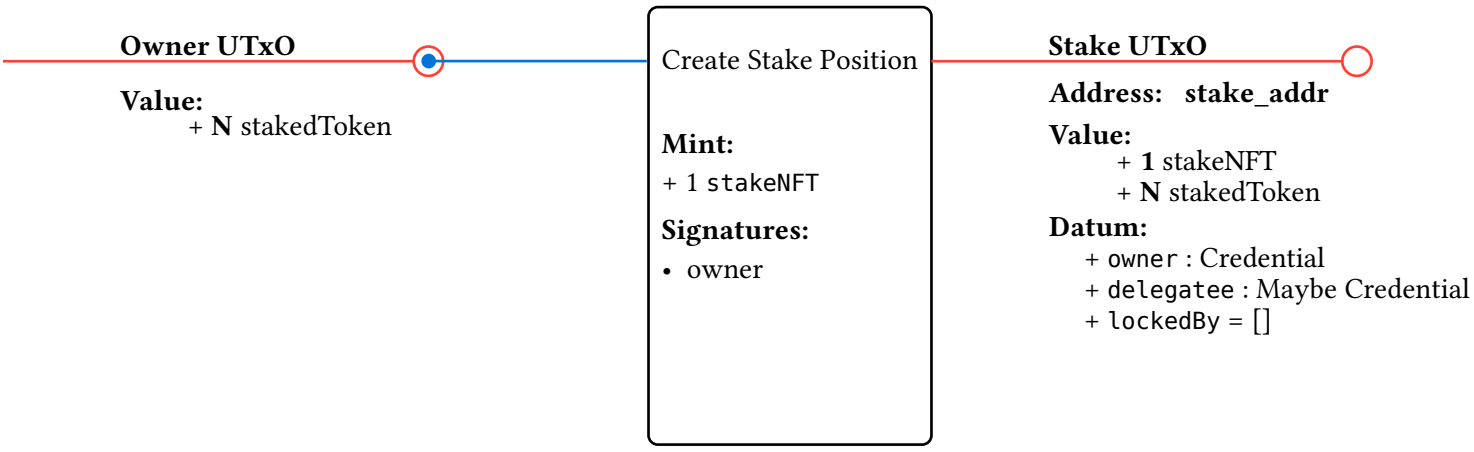
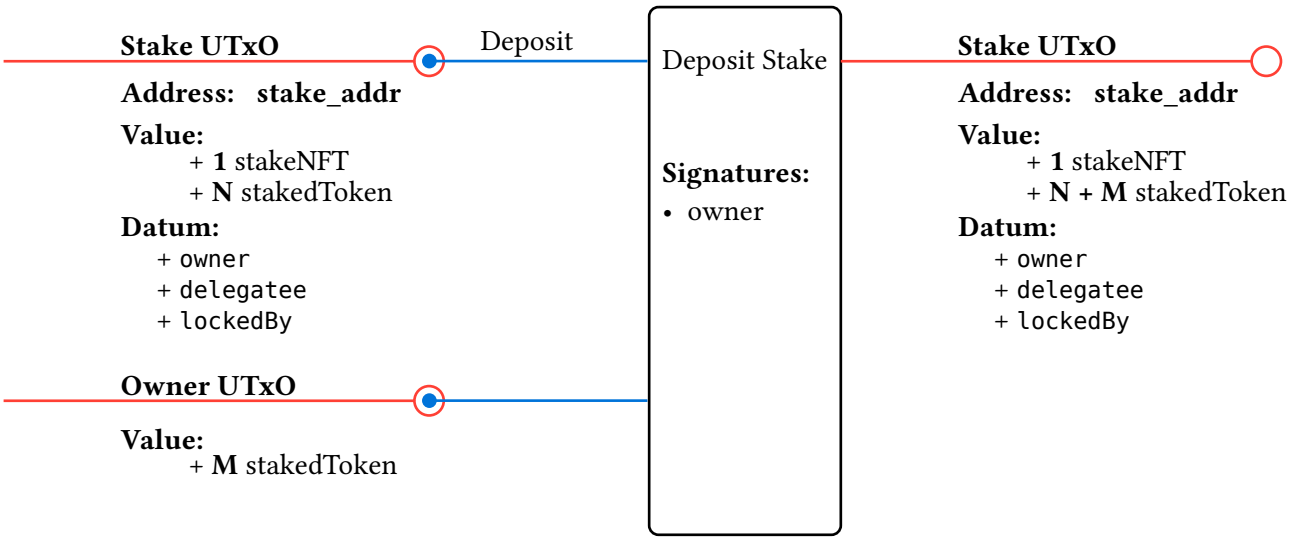
Create Stake Position

Creates a stake position UTxO with a stake NFT and the initial staked tokens.



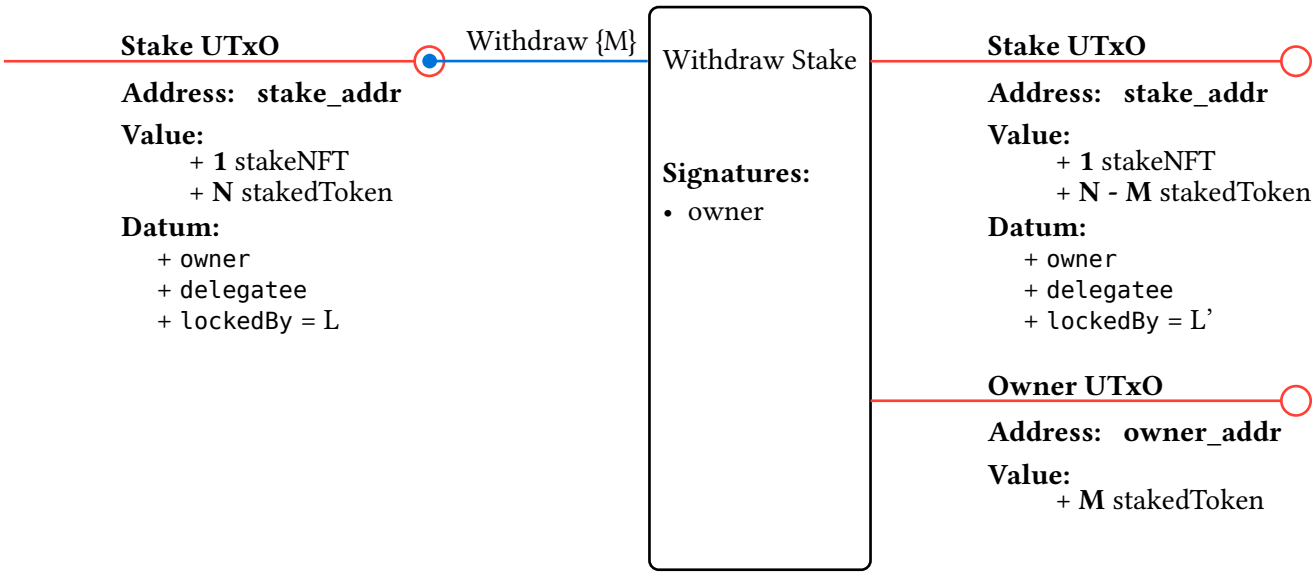
Deposit Stake

Adds more staked tokens to an existing stake position UTxO.



Withdraw Stake

Removes a portion of free staked tokens from a stake position.



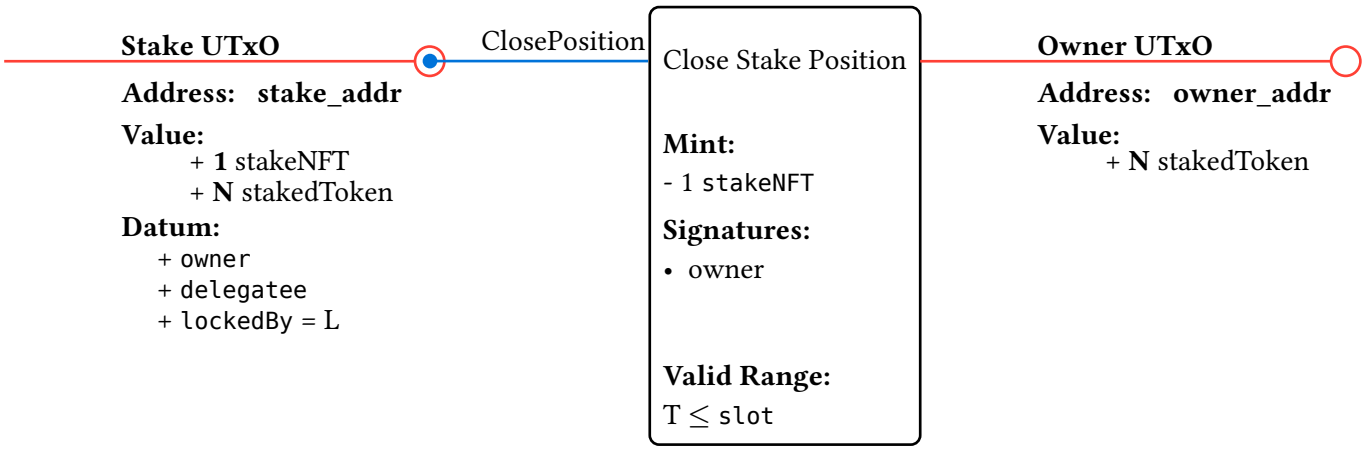
Notes:

$L' = [(\text{pId}_1, t_1, s_1), ..., (\text{pId}_n, t_n, s_n)]$

- `lockedStaked = max(s_i)`
- `freeStaked = N - lockedStaked`
- `M <= freeStaked`

Close Stake Position

Burns the stake NFT and returns all staked tokens after all locks expire.



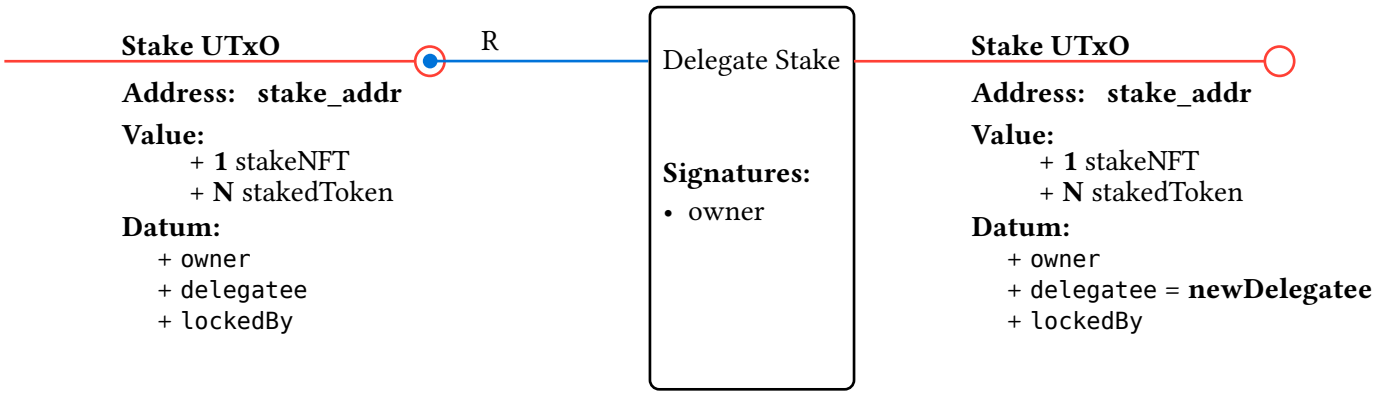
Notes:

$L = [(\text{pId}_1, t_1, s_1), ..., (\text{pId}_n, t_n, s_n)]$

$t_i \leq \text{T}\ \forall i$

Delegate Stake

Updates the delegatee of a stake position to redirect voting power.

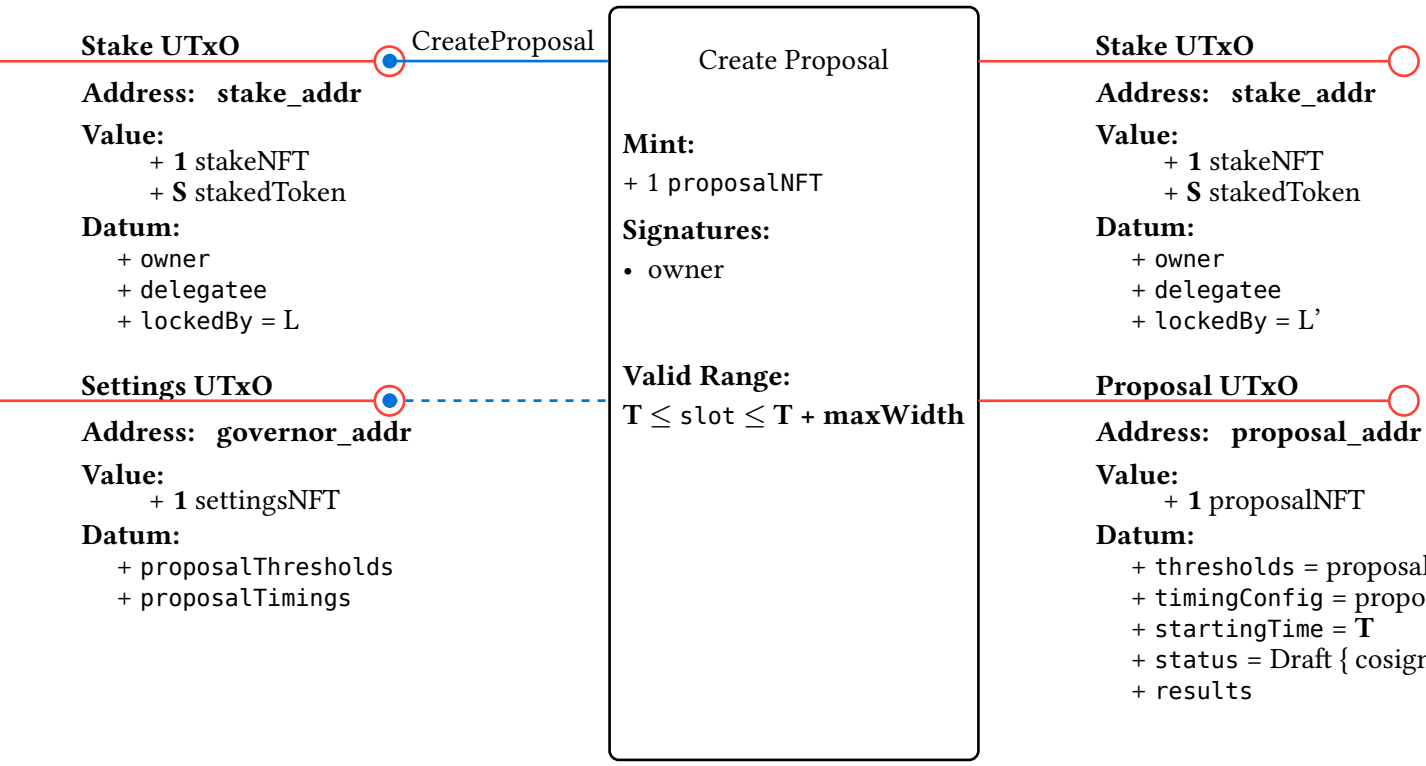


Notes:

R = DelegateTo { newDelegatee }

Create Proposal

Mints a proposal NFT and starts a new governance proposal in draft phase.

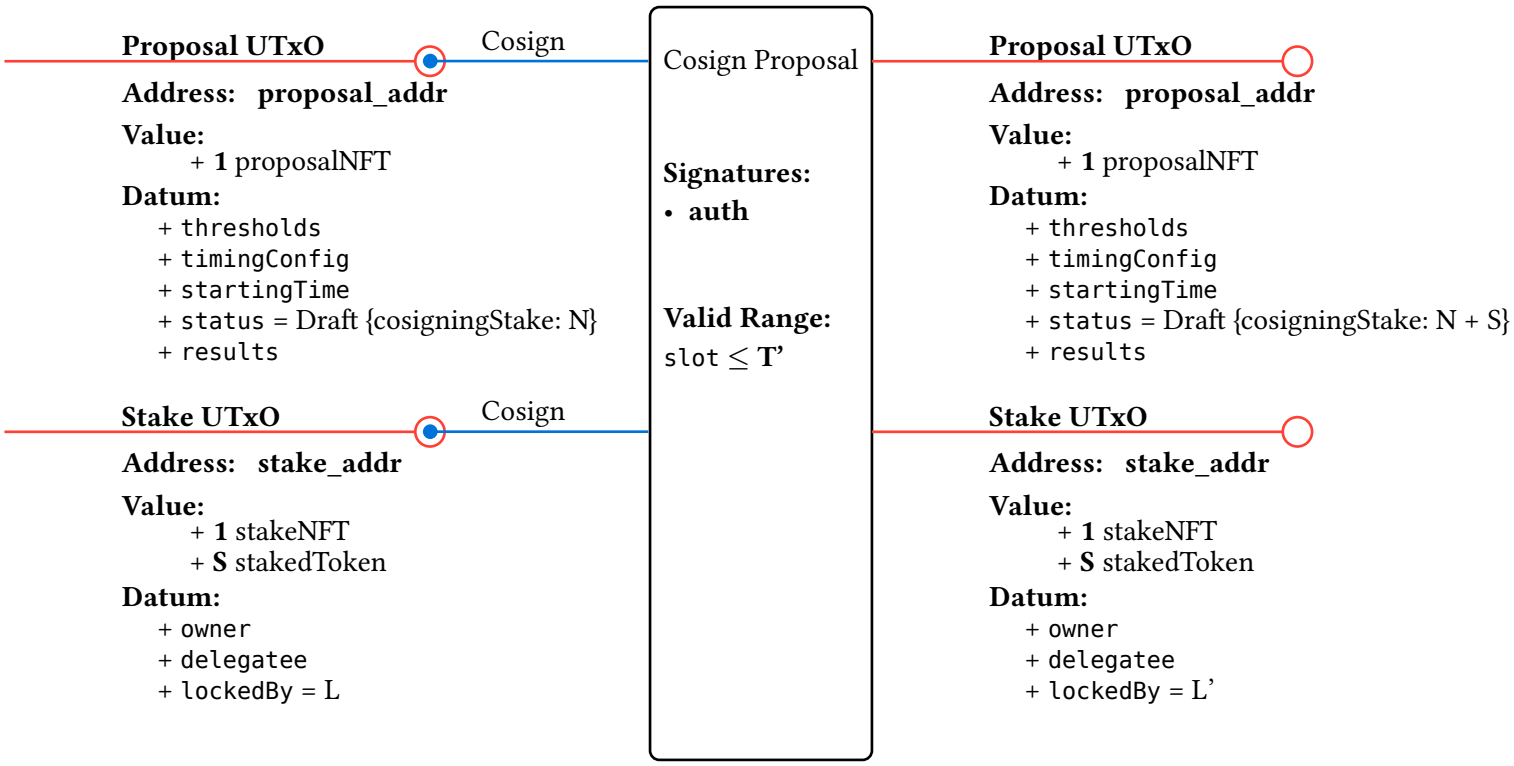


Notes:

- `pId = proposalNFT.tokenName = hash(stakeUtxo.outputReference)`
- `L' = [..L, (pId, T + timingConfig.draftlength, S)]`
- `S >= thresholds.create`

Cosign Proposal

Adds stake to a proposal's cosigning requirements during the draft phase.



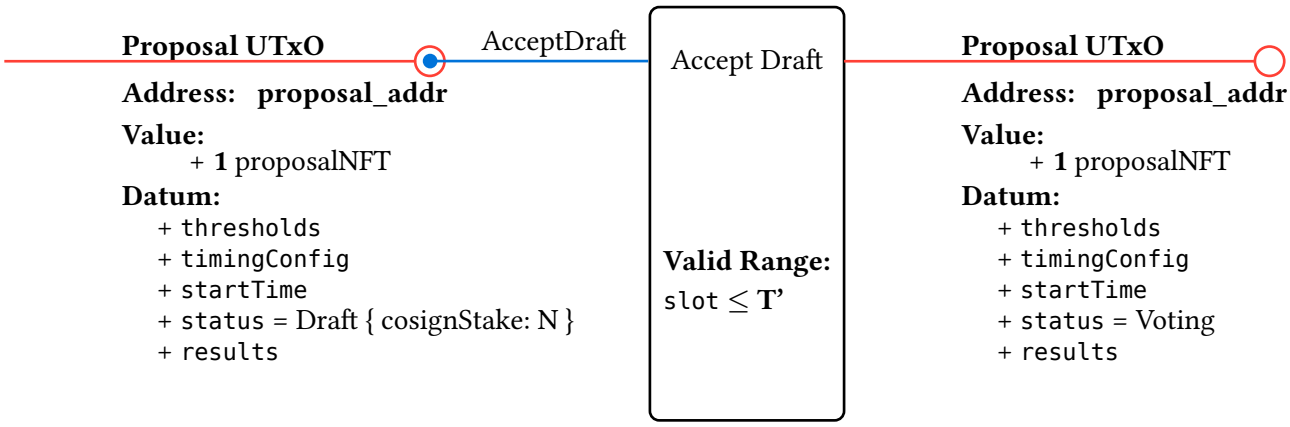
Notes:

- `pId = proposalNFT.tokenName`
- `pId not in L`
- `T' = startingTime + timingConfig.draftLength`
- `L' = [..L, (pId, T', S)]`
- `S >= thresholds.cosign`

```
auth = when delegatee is
  Some d = d
  None = owner
```

Accept Draft

Advances a proposal from draft to voting phase once co-signing thresholds are met.

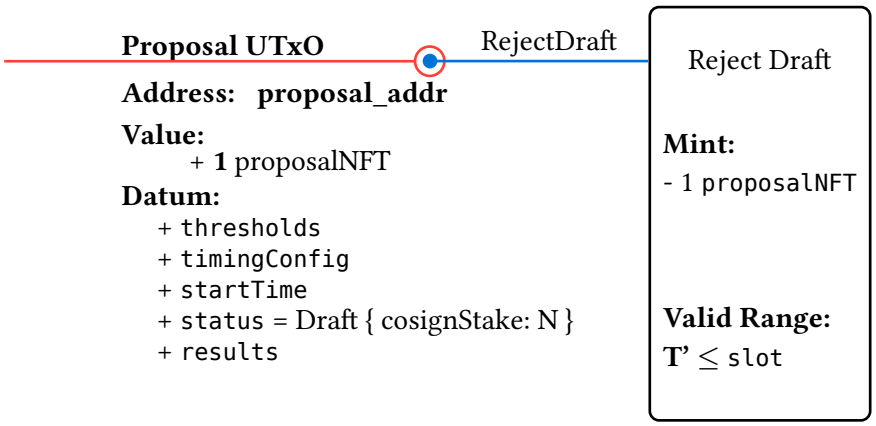


Notes:

- T' = startingTime + timingConfig.draftLength
- N >= thresholds.accept

Reject Draft

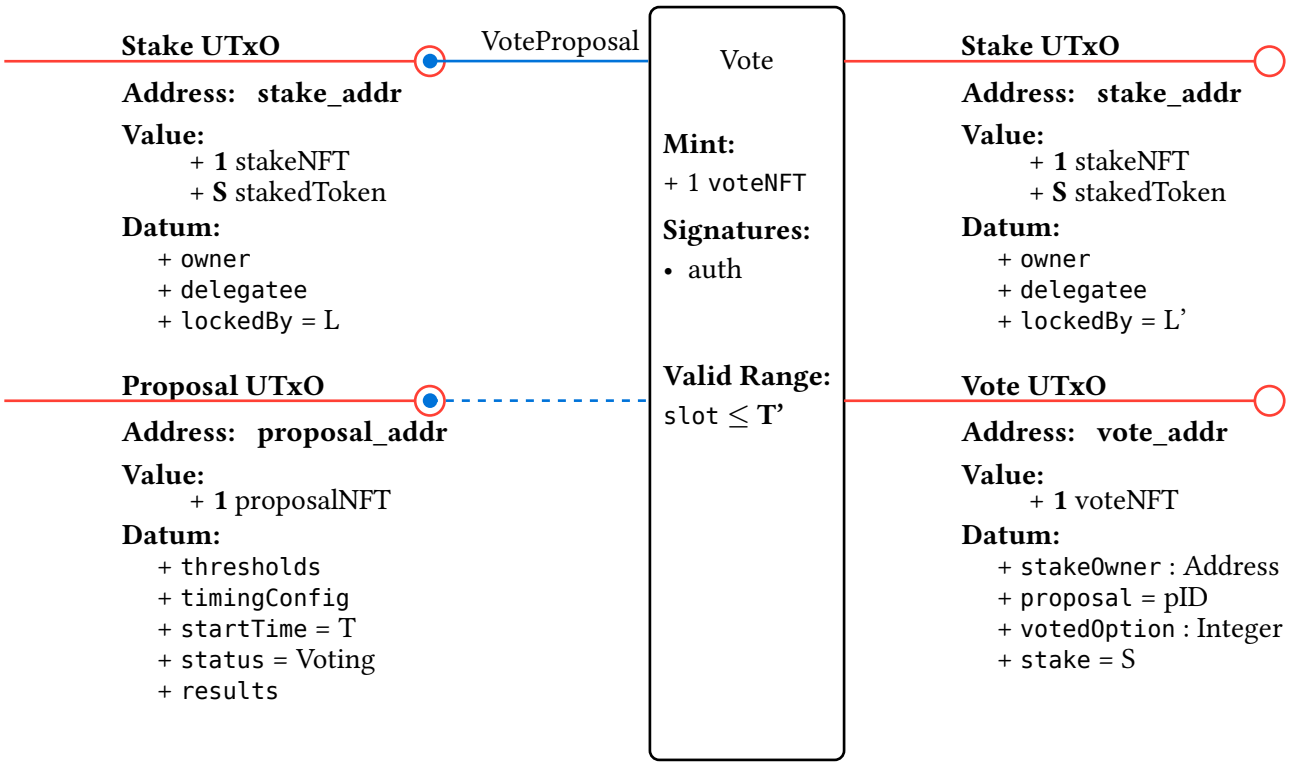
Destroys a proposal that fails to meet co-signing thresholds before the draft deadline.



Notes:
- T' = startingTime + timingConfig.draftLength

Vote

Casts a vote on a proposal by minting a vote NFT locked to the voter's choice.



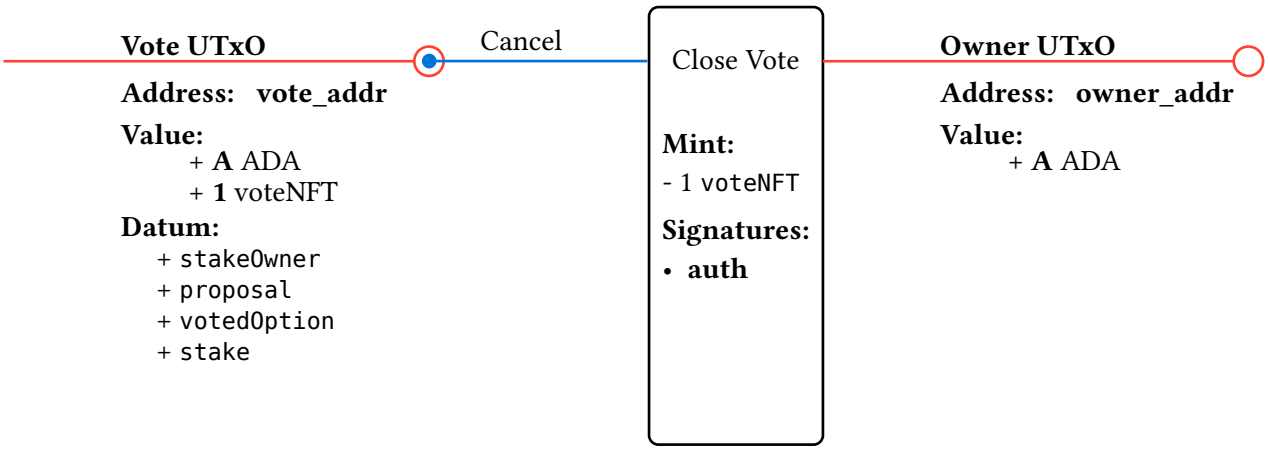
Notes:

- `pId = proposalNFT.tokenName`
- `pId not in L`
- `L' = [..L, (pId, T, S)]`
- `S >= thresholds.vote`
- `T' = T + timingConfig.draftLength + timingConfig.votingLength`

```
auth = when delegatee is
  Some d = d
  None = owner
```

Close Vote

Cancels a vote by burning its NFT, releasing the stake owner's ADA.

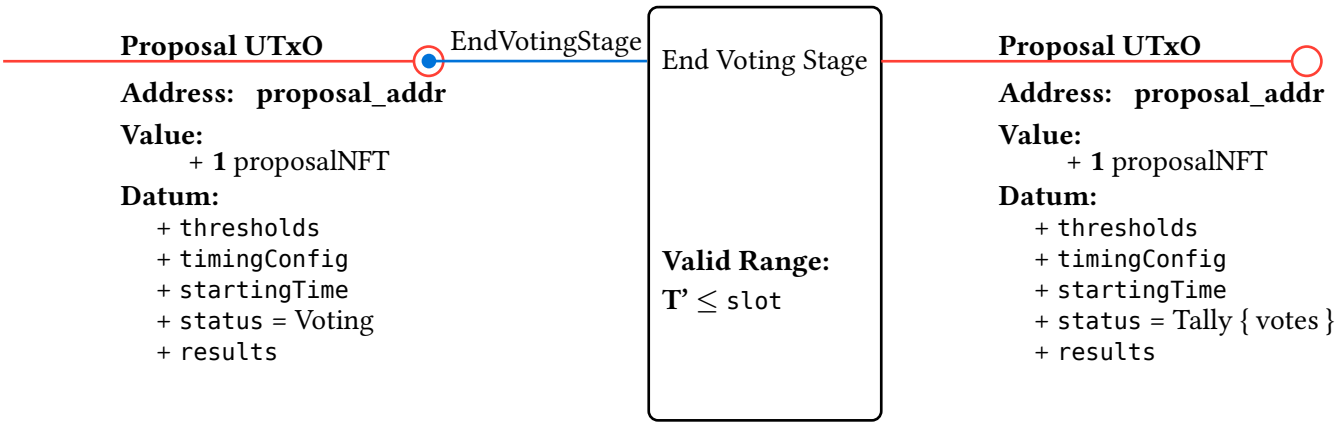


Notes:

`auth = stakeOwner`

End Voting Stage

Closes the voting period and transitions the proposal to the tally stage.

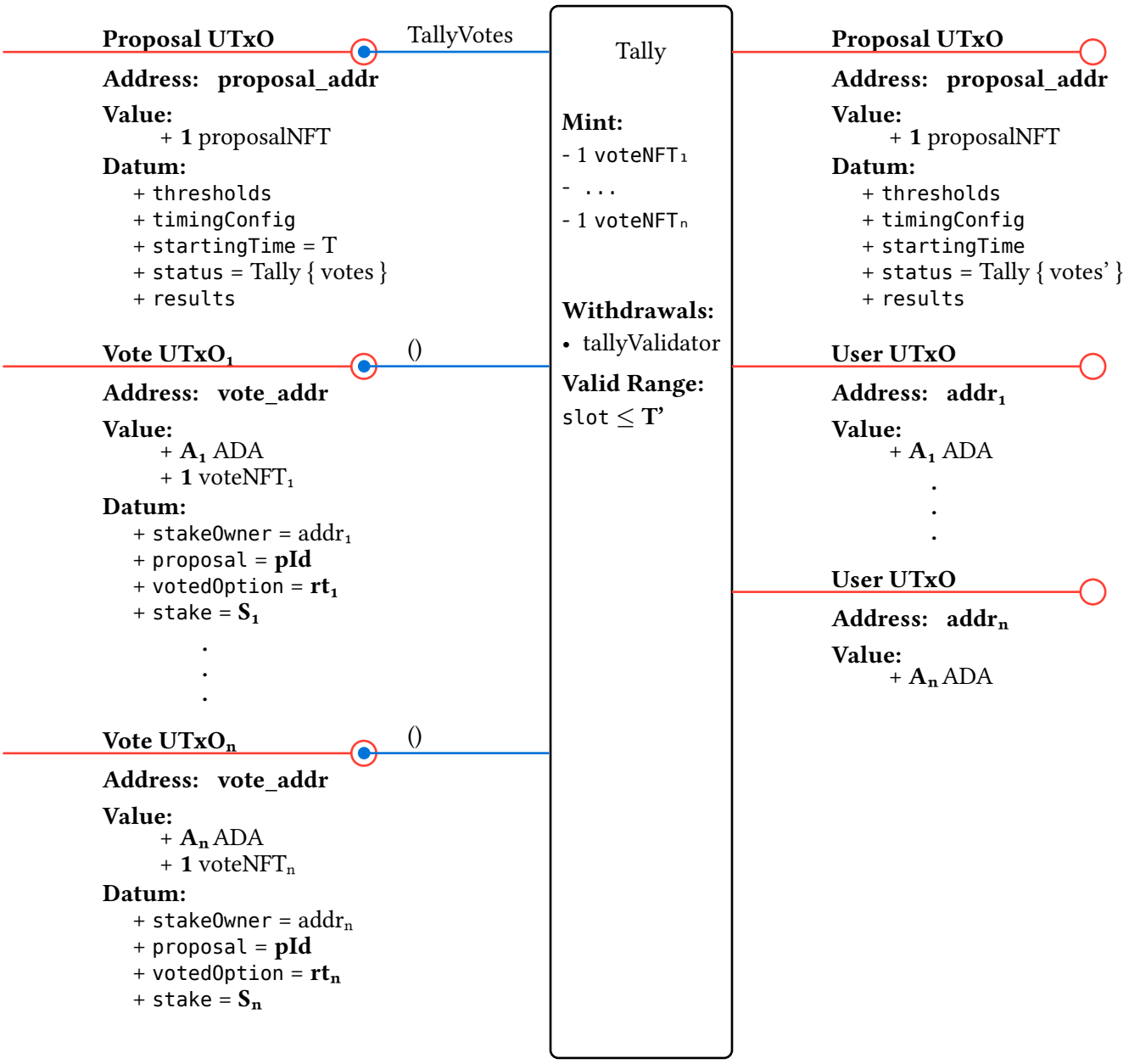


Notes:

- `T' = T + timingConfig.draftLength + timingConfig.votingLength`
- votes = {}

Tally

Counts all votes submitted for a proposal and updates the results.

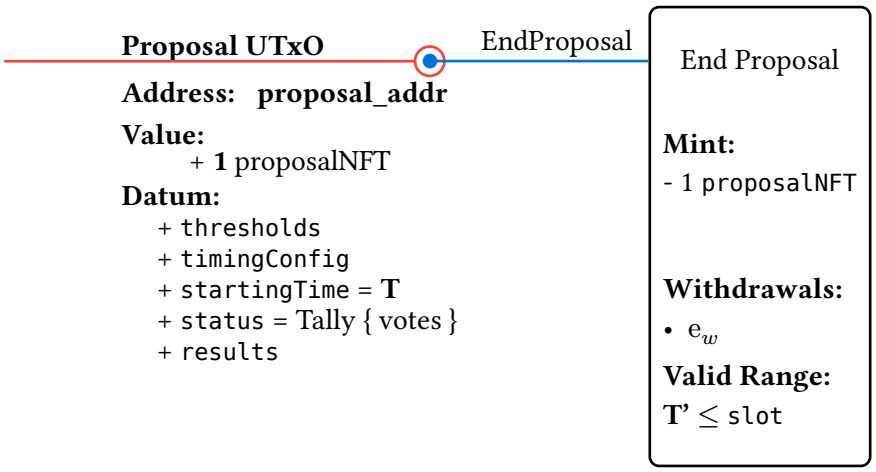


Notes:

- `pId = proposalNFT.tokenName`
- `T' = T + timingConfig.draftLength + timingConfig.votingLength + timingConfig.tallyLength`
- `votes` map get each rt_i incremented by S_i

End Proposal

Executes the winning outcome or destroys a failed proposal after the tally period.



Notes:

- T' = T + timingConfig.draftLength + timingConfig.votingLength + timingConfig.tallyLength
- votes = $\{rt_1 : v_i, ..., rt_n : v_n\}$
- mostVoted = rt_i where $v_i > v_j \forall j \neq i$
- `v_i >= thresholds.execute`
- e_w = results[rt_i]

if tie or not enough votes, just destroy UTxO (ie. e_w = None)